

```
In [11]: import pandas as pd
import numpy as np
from sklearn.metrics import r2_score, accuracy_score
from utils.data_handling import sequential_split, load_data
from utils.feature_creation import create_features
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor
```

```
In [12]: #Importing data
path = "../data/BRK.csv"
df = load_data(path)
```

```
In [13]: #Creating and visualizing feature space
features = create_features(df, classifier=False)
features = features.dropna()

X = features.drop(columns=['Close', 'Date'])
y = features['Close']

pd.set_option('display.width', 1000)
print(X)
```

	Close Day -1	Close Day -2	Close Day -3	Close Day -4	Close Day -5	Close Day -6	Close Day -7
	10MA	50MA	200MA	RSI	MACD	MACD Signal	MACD Histogram
200	133.800003	131.179993	132.190002	133.399994	133.02	133.809998	133.750000
9999	132.740999	133.553800	141.580999	54.546370	0.141387	0.5102	33
							-0.368846
201	133.809998	133.800003	131.179993	132.190002	133.39	133.619995	133.029999
9994	132.864998	133.355200	141.503249	56.231804	0.232260	0.4808	84
							-0.248624
202	133.619995	133.809998	133.800003	131.179993	132.19	134.160004	133.399994
0002	133.161998	133.170000	141.439049	52.577095	0.343887	0.4740	10
							-0.130122
203	133.619995	133.809998	133.800003	131.179993	131.17	133.220001	132.190002
9993	133.215999	132.978400	141.370950	63.606340	0.352440	0.3860	50
							-0.033610
204	133.220001	134.160004	133.619995	133.809998	133.80	137.289993	131.179993
0003	133.569998	132.892200	141.312999	64.671644	0.679796	0.5707	25
							0.109072
...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...
2403	434.470001	441.820007	445.609985	438.899994	434.42	435.980011	424.440002
0013	429.873001	413.807000	387.474600	63.409721	7.876475	2.5653	03
							5.311172
2404	434.470001	441.820007	445.609985	438.89	434.010010	435.980011	434.010010
9994	432.222003	414.310800	387.926200	61.772825	7.689208	1.9024	29
							5.786779
2405	434.010010	435.980011	434.470001	441.820007	445.60	432.799988	438.899994
9985	434.123001	414.725800	388.364900	62.198405	7.358337	1.2572	47
							6.101091
2406	434.010010	435.980011	434.470001	441.82	433.290009	432.799988	433.290009
0007	435.574002	415.167200	388.799650	65.849759	7.054342	0.7626	01
							6.291741
2407	434.010010	435.980011	434.470001	441.82	437.660004	433.290009	432.799988
0001	436.896002	415.715601	389.260700	66.370093	7.084381	0.6341	12
							6.450269

[2208 rows x 14 columns]

```
In [14]: #Initialize ML model regressors
lin_model = LinearRegression()
rfmodel = RandomForestRegressor(n_estimators=1000, max_depth=10, random_state=42)
knn_model = KNeighborsRegressor(n_neighbors = 5)

In [15]: fold_length = np.floor(len(X)/10).astype(int) #Assigns 220 to fold_length
training_length = (np.floor(len(X)/10)*5).astype(int) #Assigns 1100 to training_L

lin_acc_list = [] #Empty list to store directional accuracies of linear model
rf_acc_list = [] #Empty list to store directional accuracies of random forest mo
knn_acc_list = [] #Empty list to store directional accuracies of knn model

In [16]: '''Perform a rolling window cross validation where initial 50% of data is used for
the next 10% is used for testing. On iteration 2, data range from 10%-60% is use
and range from 60% - 70% is used for testing, and so on.. '''
i = 0
```

```

for j in range(1, 6):
    training_subset_X = X[(i*fold_length):training_length+(i*fold_length)]
    training_subset_y = y[(i*fold_length):training_length+(i*fold_length)]
    testing_subset_X = X[training_length+(fold_length*i):training_length+(fold_leng
    testing_subset_y = y[training_length+(fold_length*i):training_length+(fold_leng

    #Fitting and testing 5 different data subsets for the linear model
    lin_model.fit(training_subset_X, training_subset_y)
    validation_set_predictions = lin_model.predict(testing_subset_X)
    predicted_direction = (np.diff(validation_set_predictions) > 0).astype(int)
    actual_direction = (np.diff(testing_subset_y) > 0).astype(int)
    dir_acc = accuracy_score(actual_direction, predicted_direction) * 100
    lin_acc_list.append(dir_acc)

    #Fitting and testing 5 different data subsets for the random forest model
    rfmodel.fit(training_subset_X, training_subset_y)
    validation_set_predictions = rfmodel.predict(testing_subset_X)
    predicted_direction = (np.diff(validation_set_predictions) > 0).astype(int)
    actual_direction = (np.diff(testing_subset_y) > 0).astype(int)
    dir_acc = accuracy_score(actual_direction, predicted_direction) * 100
    rf_acc_list.append(dir_acc)

    #Fitting and testing 5 different data subsets for the knn model
    knn_model.fit(training_subset_X, training_subset_y)
    validation_set_predictions = knn_model.predict(testing_subset_X)
    predicted_direction = (np.diff(validation_set_predictions) > 0).astype(int)
    actual_direction = (np.diff(testing_subset_y) > 0).astype(int)
    dir_acc = accuracy_score(actual_direction, predicted_direction) * 100
    knn_acc_list.append(dir_acc)

    i+=1

```

```

In [17]: #Calculate and print all mean directional accuracies
print(f'Linear Model Mean Directional Accuracy: {round(np.mean(lin_acc_list), 3)}%')
print(f'Random Forest Model Mean Directional Accuracy: {round(np.mean(rf_acc_list),
print(f'KNN Model Mean Directional Accuracy: {round(np.mean(knn_acc_list), 3)}%')

```

Linear Model Mean Directional Accuracy: 72.968%
Random Forest Model Mean Directional Accuracy: 70.32%
KNN Model Mean Directional Accuracy: 62.192%

```

In [18]: import matplotlib.pyplot as plt

#Simple bar graph to display the mean directional accuracy
def plot_bar(x, mean_accuracy):
    plt.bar(x, mean_accuracy, color = 'orange')

#Calculate 95% confidence interval using z score of 1.96 and format graph to displa
#for visual comparison of model performance
def plot_ci(x, accuracies, z = 1.96, color='blue', horizontal_line_width = 0.3):
    mean = np.mean(accuracies)
    st_dev = np.std(accuracies)
    confidence_interval = z * st_dev / np.sqrt(len(accuracies))

    left_margin = x - horizontal_line_width / 2
    right_margin = x + horizontal_line_width / 2

```

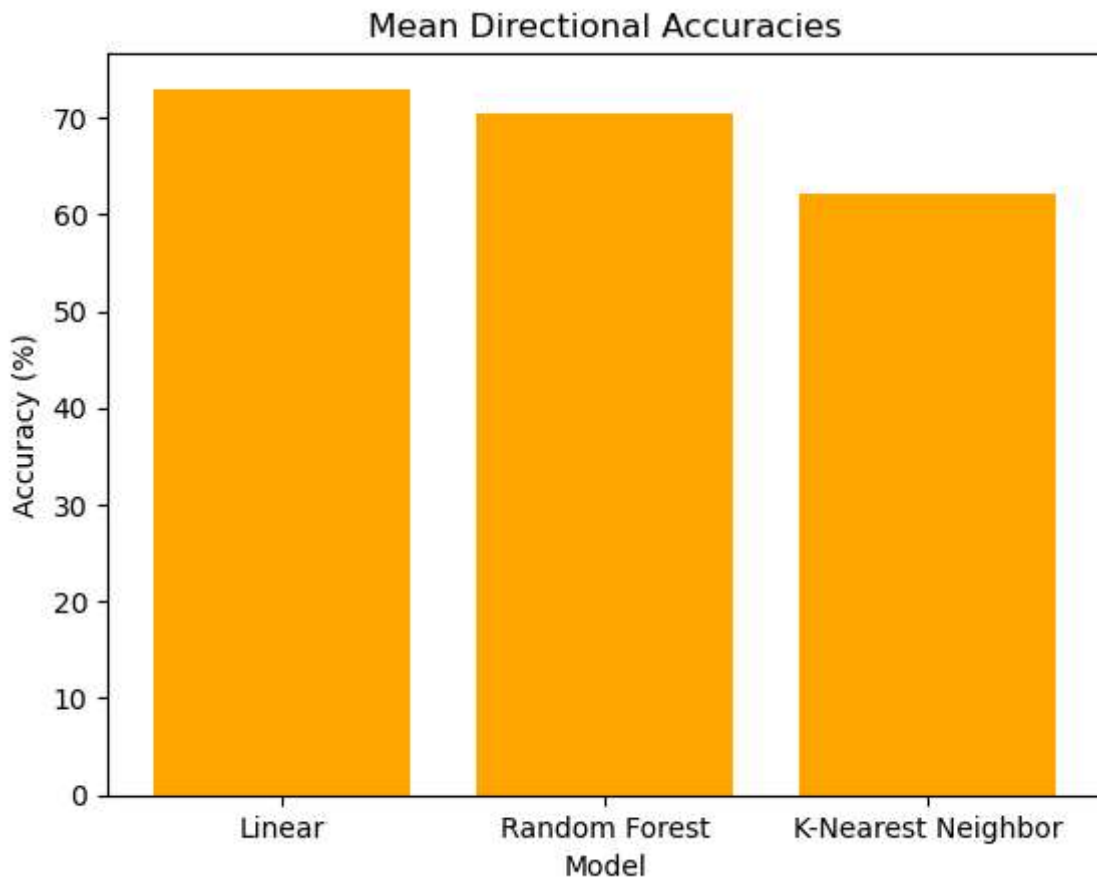
```

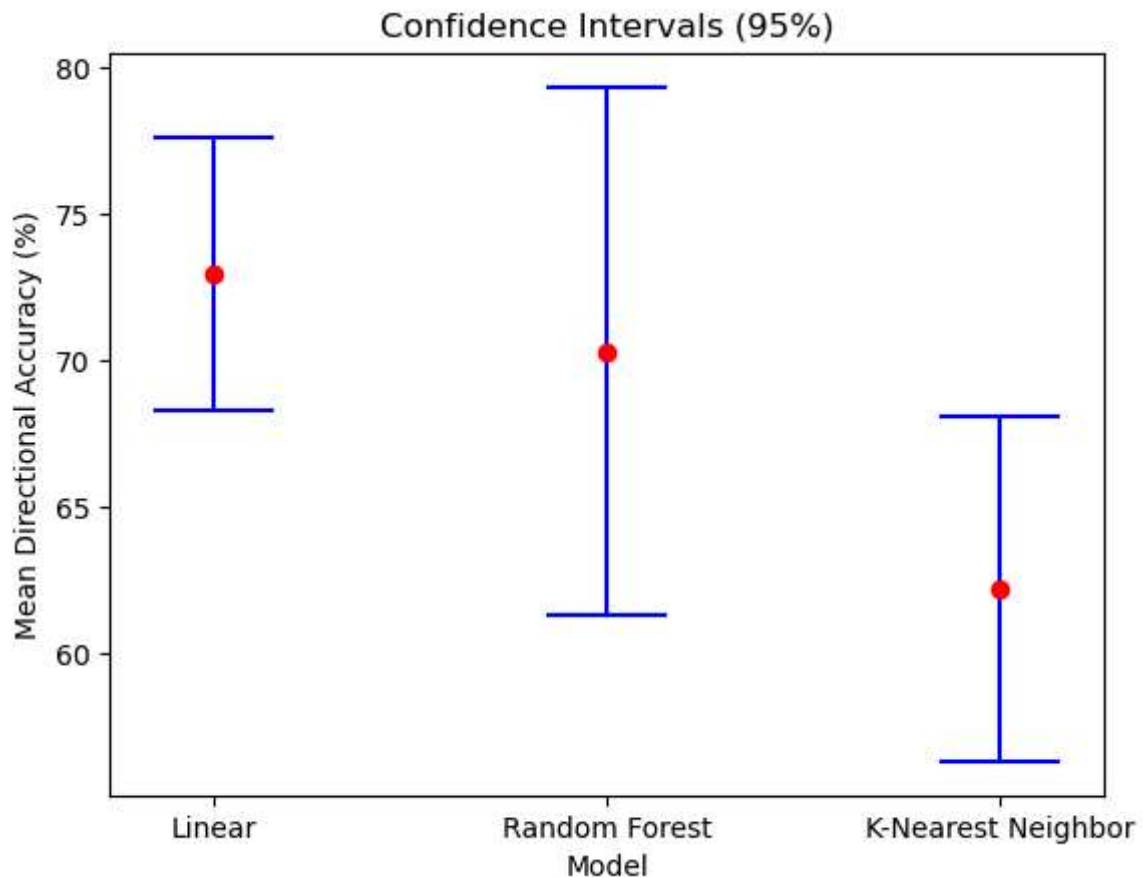
upper_bound = mean + confidence_interval
lower_bound = mean - confidence_interval
plt.plot([x, x], [upper_bound, lower_bound], color=color)
plt.plot([left_margin, right_margin], [upper_bound, upper_bound], color=color)
plt.plot([left_margin, right_margin], [lower_bound, lower_bound], color=color)
plt.plot(x, mean, 'o', color='red')

#Create and Display Bar Graph of Mean Directional Accuracies
plt.title('Mean Directional Accuracies')
plot_bar('Linear', np.mean(lin_acc_list))
plot_bar('Random Forest', np.mean(rf_acc_list))
plot_bar('K-Nearest Neighbor', np.mean(knn_acc_list))
plt.xlabel('Model')
plt.ylabel('Accuracy (%)')
plt.show()

#Create and Display Graph of Confidence Intervals
plt.xticks([1, 2, 3], ['Linear', 'Random Forest', 'K-Nearest Neighbor'])
plt.title('Confidence Intervals (95%)')
plot_ci(1, lin_acc_list)
plot_ci(2, rf_acc_list)
plot_ci(3, knn_acc_list)
plt.xlabel('Model')
plt.ylabel('Mean Directional Accuracy (%)')
plt.show()

```





```
In [19]: print('Team Member Contribution')
print()
print('David T. Wilkey:')
print('\tOriginal linear regression model development')
print('\tRolling window cross validation and confidence interval calculations')
print('\tCreated/added poster pictures (rolling window cross validation illustration)')
print('\tAssist with poster writing (methodology, results, part of lessons) and generate')
print()
print('Bhumir Patel:')
print('\tOriginal K-nearest neighbor model development')
print('\tCoordinated poster organization and printing')
print('\tManual code review and accuracy verifications')
print('\tAssist with poster writing (abstract, motivation, most of lessons, find/or generate')
print()
print('Joey Syracuse:')
print('\tOriginal random forest model development')
print('\tUtility and feature creation/splitting functions')
print('\tCreated/added poster pictures (train test split image, stock brokers image)')
print('\tAssist with poster writing (methodology, results, part of lessons) and generate')
```

Team Member Contribution

David T. Wilkey:

- Original linear regression model development
- Rolling window cross validation and confidence interval calculations
- Created/added poster pictures (rolling window cross validation illustration and k-fold with red X illustration were created in MS Paint, confidence intervals, and mean accuracy images)
- Assist with poster writing (methodology, results, part of lessons) and general review/edits

Bhumir Patel:

- Original K-nearest neighbor model development
- Coordinated poster organization and printing
- Manual code review and accuracy verifications
- Assist with poster writing (abstract, motivation, most of lessons, find/organize references) and general review/edits

Joey Syracuse:

- Original random forest model development
- Utility and feature creation/splitting functions
- Created/added poster pictures (train test split image, stock brokers image, AAPL comparison image)
- Assist with poster writing (methodology, results, part of lessons) and general review/edits